Project Design Phase-II

Data Flow Diagram & User Stories

Team Leader : VIJAYALAKSHMI A

Team Memebers:

SWETHA V

BRINDHA S V

BHAVANA G

Date	07/03/2026
Team ID	SWTID-2026-3353
Project Name	Product Inventory Management

Product Inventory Management System

PROJECT OVERVIEW

The **Product Inventory Management System** is a backend-focused web application developed using **Node.js**, **Express.js**, and **MongoDB**, designed to provide a secure and scalable solution for managing product data through RESTful APIs. The system enables users to create accounts, log in securely, and access protected routes using **JWT-based authentication**, ensuring that only authorized users can perform operations. It supports complete **CRUD functionality**, allowing users to add new products, view their inventory, update product details such as price or quantity, and delete items when needed.

Key Objectives:

• Provide secure JWT-based user authentication and authorization for protected API access.

• Enable efficient product inventory management through RESTful CRUD operations.

• Ensure user-specific data isolation so each user can manage only their own products.

• Implement scalable backend architecture using Node.js, Express.js, and MongoDB.

• Deliver fast and reliable API responses with proper validation and error handling.

Target Users:

• Small business owners managing product stock and inventory records.

• Freelancers or sellers tracking items or services in a structured way.

• Developers learning backend API development and authentication workflows.

• Students working on backend or full-stack development projects.

• Anyone needing a secure and simple backend system for managing product data.

PROJECT INSTRUCTION

• Create backend project folder
• Configure server using Express.js
• Setup MongoDB database connection
• Implement authentication APIs
• Implement product CRUD APIs
• Test APIs using Thunder Client in VS Code

PRE-REQUISITES

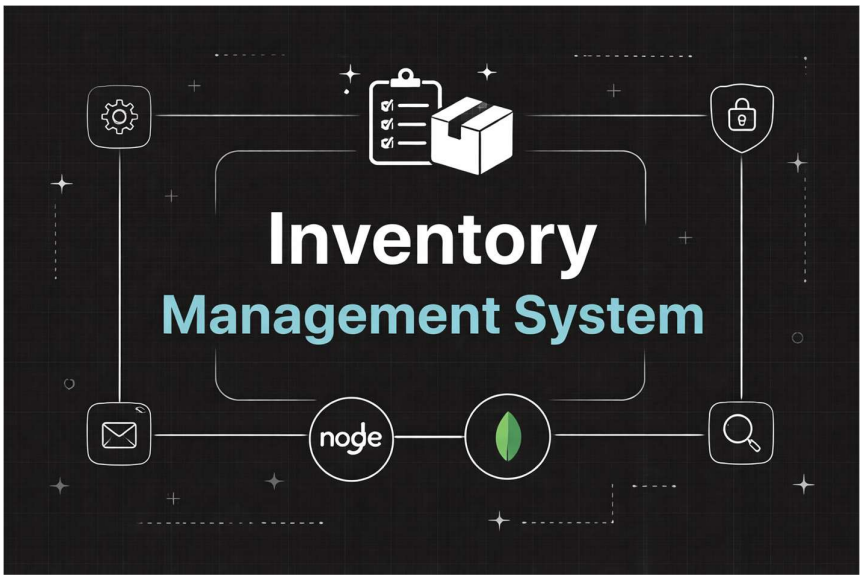
Software	**Purpose**
Node.js	JavaScript runtime for backend
MongoDB	Database storage
npm	Package manager
VS Code	Development environment

Knowledge Requirements

• JavaScript ES6 concepts
• Node.js and Express.js REST API development
• MongoDB and Mongoose ODM

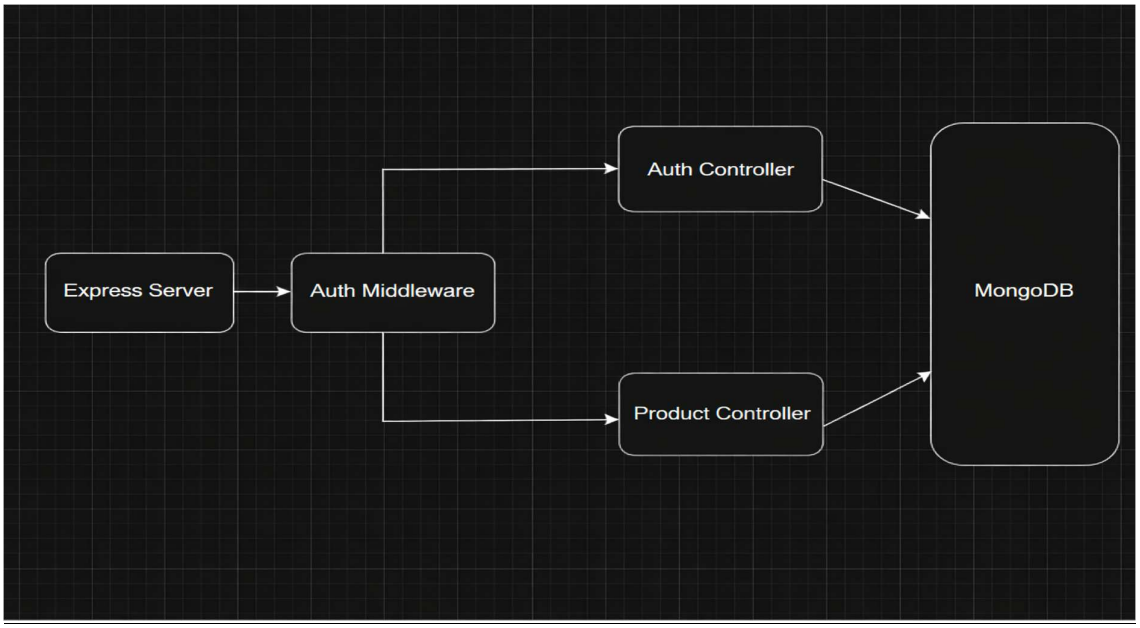
- JWT authentication
- Middleware architecture

Project Thumbnail:



PROJECT ARCHITECTURE

High-level architecture diagram illustrating the three-tier backend structure.



Node.js + Express Backend

- RESTful API architecture
- JWT authentication middleware

• MVC pattern (Models, Controllers, Routes)

• Input validation and error handling

MongoDB Database

• Document-based NoSQL storage

• Mongoose ODM for schema modeling

• Two collections: Users and Contacts

• Indexed fields for optimized queries

TECHNICAL ARCHITECTURE

Backend Architecture (Node.js + Express)

- **Controllers**: Handle authentication and product logic
- **Models**: Define database schema
- **Routes**: API endpoints for authentication and products
- **Middleware**: JWT verification and error handling
- **Configuration**: Environment variables and database connection

Database Architecture (MongoDB)

Users Collection:

- _id
- name
- email
- password (hashed)
- timestamps

Products Collection:

- _id
- userId (reference to user)
- name
- price
- quantity
- description
- timestamps

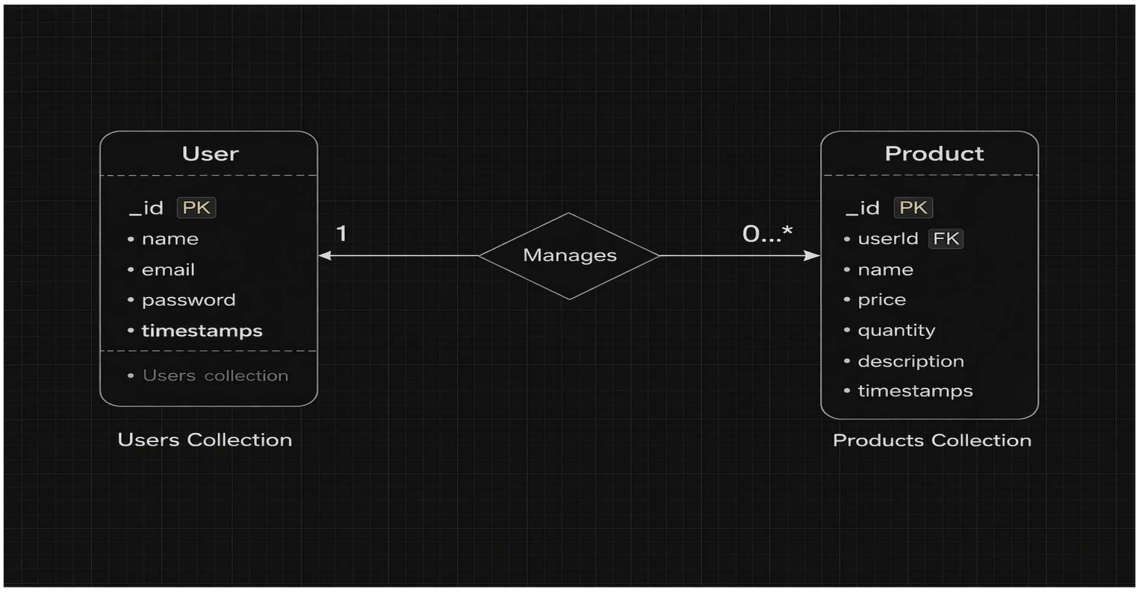
Relationship: One User → Many Products

Indexes:

- email (unique)
- userId
- product name

ER Diagram

ER diagram showing User → Product relationship:



Features:

Core Features

S.No	Feature	Description
1	User Authentication	JWT-based register and login system
2	Product CRUD	Add, view, update and delete products
3	Private Data Access	Users access only their own products
4	Product Search	Search products by name
5	Secure API	Middleware protected routes

Roles and Responsibility

User

Registration: Register using name, email and password

Login: Login using registered credentials

Add Product: Create product with name, price, quantity and description

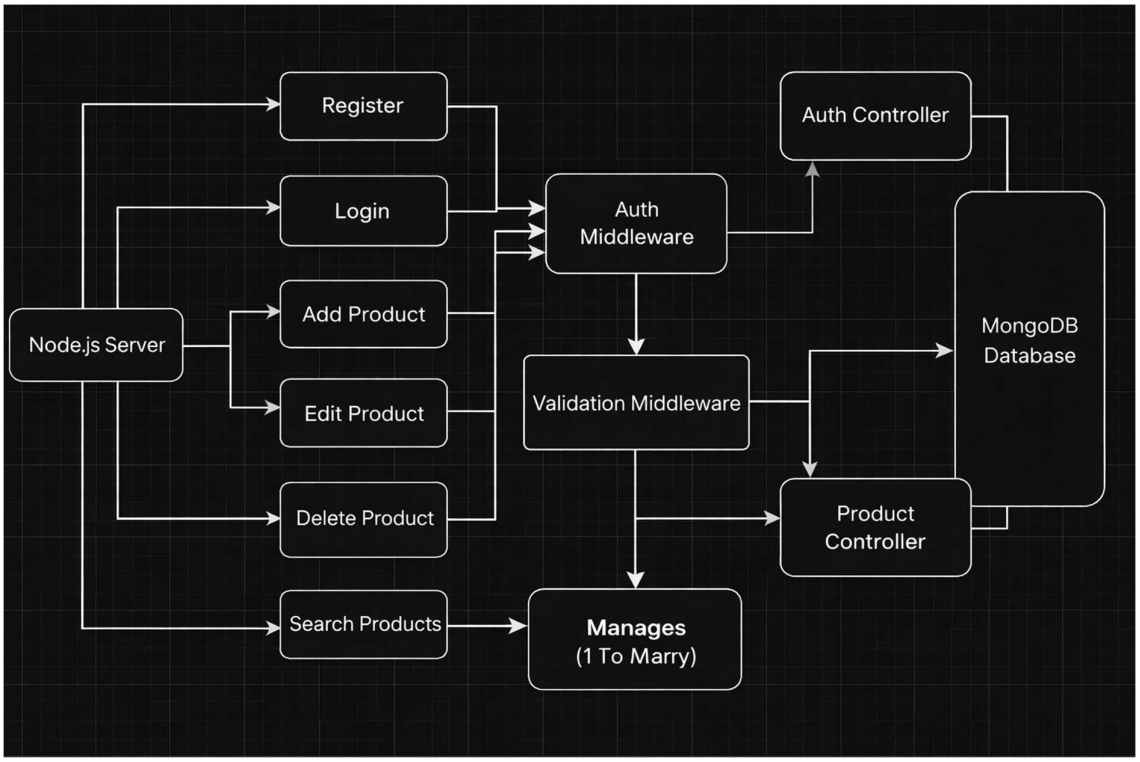
Edit Product: Update existing product

Delete Product: Remove product from inventory

Search Product: Search products using keyword

User Flow

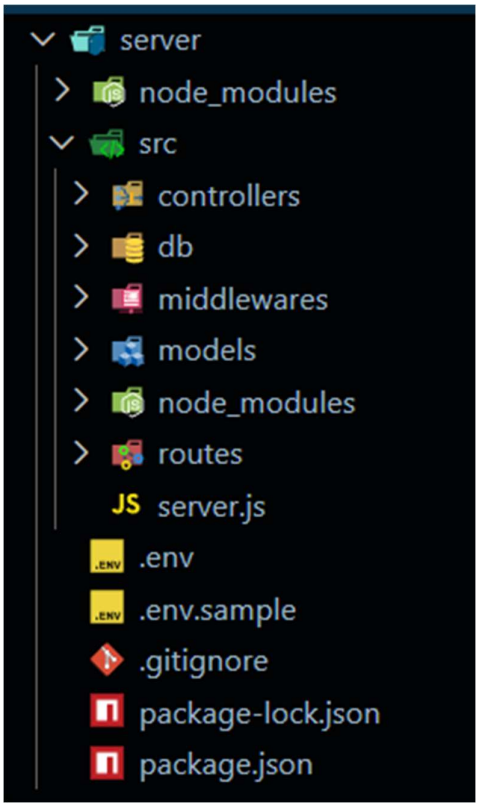
1. The user sends a register/login request.
2. The backend generates a JWT token.
3. The user sends a token in the Authorization header.
4. Middleware verifies the token.
5. The controller performs CRUD operations.
6. MongoDB stores or retrieves product data.
7. API sends response.



Project Setup and Configuration

Folder Structure

```
server/
│
├── node_modules/
├── src/
│   ├── controllers/
│   ├── db/
│   ├── middlewares/
│   ├── models/
│   ├── routes/
│   └── server.js
│
├── .env
├── .gitignore
├── package-lock.json
└── package.json
```



Initialize Node.js Project:

Open terminal inside the server folder and run:

```
npm init -y
```

This creates the package.json file which manages project dependencies.

Install Required Packages:

Install backend dependencies:

```
npm install express mongoose cors dotenv jsonwebtoken bcryptjs
```

Package Purpose:

- express → Backend framework
- mongoose → MongoDB connection
- cors → Cross-origin requests

- dotenv → Environment variables
- jsonwebtoken → Authentication
- bcryptjs → Password hashing

Create Environment File:

Create a .env file in the root of the server folder.

Example:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
```

Backend Development

Setup Express Server:

Inside src/server.js, create the basic Express server.

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

Create Controllers

Inside: src/controllers/

Create:

auth.controller.js

```
export const registerUser = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};


export const loginUser = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Auth Controllers

The Auth Controller manages all user authentication-related operations in the system. It handles user registration and login functionality while ensuring secure access using JWT tokens.

Responsibilities:

- Register new users
- Validate user input data
- Hash passwords using bcrypt before storing
- Authenticate users during login
- Generate JWT tokens for authorized access
- Send appropriate success and error responses

Main Functionalities:

- Register User: Creates a new user account after validating email uniqueness and encrypting the password.
- Login User: Verifies credentials and returns a JWT token for protected API access.

product.controller.js

```
export const createProduct = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const getProducts = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const updateProduct = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const deleteProduct = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Product Controller

The **Product Controller** handles all product inventory operations and contains the business logic for managing products linked to individual users.

Responsibilities:

- Add new products to inventory
- Fetch products belonging to logged-in users
- Update product details like price, quantity, or description
- Delete products securely
- Perform product search functionality

Main Functionalities:

- **Create Product:** Adds a new product with a reference to the authenticated user.
- **Get Products:** Retrieves only the products created by the logged-in user.
- **Update Product:** Modifies product details while verifying ownership.

- **Delete Product:** Removes a product safely from the database.
- **Search Product:** Allows filtering products based on name.

Create Middleware

Inside: src/middlewares/

Create:auth.middleware.js

```
export const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Not authorized" });
  }

  try {
    const token = authHeader.split(" ")[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ message: "Token invalid" });
  }
};
```

Auth Middleware (protect)

The **Auth Middleware** secures protected routes by verifying the JSON Web Token (JWT) sent in the request header. It checks for a valid Authorization header starting with "Bearer " and validates the token using the secret key. If the token is valid, the decoded user information is attached to req.user and the request proceeds to the next controller. If the token is missing or invalid, the middleware returns a **401 Unauthorized** response, preventing access to private APIs.

Create Routes

Inside: src/routes/

Create:

auth.routes.js

```
const authRoutes = express.Router();


authRoutes.post("/register", registerUser);

authRoutes.post("/login", loginUser);


export default authRoutes;
```

Auth Routes:

Auth routes handle user authentication by providing endpoints for registration and login. These routes connect incoming requests to the Auth Controller to create new users and generate JWT tokens for secure access.